



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Compiler Design



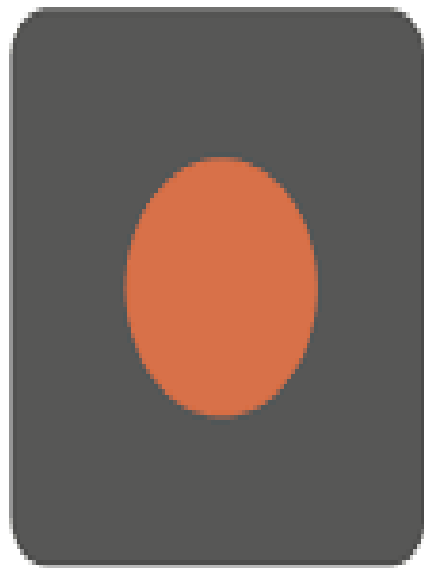
BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Contact Session-4

Syntax Analysis

IMP Note to Self



Start
Recording

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
 - Once you join the session, continue till the end to consider you as present in the class.
 - IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
 - **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**
-

Objectives

To Understand
Context - Free Grammar
Derivations – LMD, RMD
Ambiguity
First and Follow Sets

Syntax Analysis & Parsing

Just to recapitulate the definitions:

1. *Syn-tax*

The way in which words are stringed together to form phrases, clauses and sentences

2. **Syntax analysis:**

The task concerned with fitting a sequence of tokens into a specified syntax.

3. **Parsing:**

To break a sentence down into its component parts of speech with an explanation of the form, function, and syntactical relationship of each part

Syntax Analysis



The step two of Front end in Compilation

Every programming Language has RULES that prescribe the syntactic structure (i.e. the way in which words and phrases can be organized to form sentences.)

These rules can be formally specified using :

CONTEXT FREE GRAMMARS

(or GRAMMAR for Short)



Intro to Context Free Grammar (CFG)

Essential formalism for describing the structure of the programs in a programming Language

A recipe for constructing elements of a set of strings of symbols (or tokens)

Consists of a set production rules and a start symbol

Each production rule:

- Defines a named syntactic construct using Two parts
 1. An LHS (with a syntactic Construct) and
 2. An RHS (showing the possible form of the construct)
- Connected by an Arrow symbol (meaning “ is defined as”)

sentence $\longrightarrow$ *subject predicate* ‘.’

Importance of the Grammar

Grammar offers significant advantages to both language Designers and Compiler writers:

1. Gives precise, yet easy-to-understand, syntactic specification of Programming Languages
2. Helps construct efficient parsers automatically (not in all cases – of course)
3. Provides a good structure to the language which in turn helps generate correct object code
4. Makes language open ended easily amenable to add additional constructs at a later date

Context Free Grammar – Formal Definition

A context-free grammar $G = (T, N, S, P)$ consists of:

1. T , a set of *terminals* (scanner tokens - symbols that may not appear on the left side of rule,).
2. N , a set of *nonterminals* (syntactic variables generated by productions – symbols on left or right of rule).
3. S , a designated *start* nonterminal.
4. P , a set of *productions (Rules)*. Each production has the form, $A ::= \alpha$, where A is a nonterminal and α is a *sentential form*, i.e., a string of zero or more grammar symbols (terminals / nonterminals)

CFG – An Example



A grammar derives strings by

1. Beginning with the Start Symbol and
2. Repeatedly replacing a nonterminal by the right side of the production rule for that nonterminal

Example:

$expr \rightarrow expr\ op\ expr$

$expr \rightarrow (expr)$

$expr \rightarrow - expr$

$expr \rightarrow id \mid num$

$op \rightarrow [+, -, *, /, \uparrow]$

- The terminal symbols are **id, num, (,), +, -, *, /, $\uparrow$**
- The non-terminal symbols are *expr op*
- The start symbol is *expr*

Notational Conventions

1. These Symbols are terminals
 1. Lower case letters early in the alphabet (like a, b, c)
 2. Operator symbols such as +, - etc..
 3. Punctuation symbols like ,(comma), (,), etc
 4. The digits 0, 1,9
 5. Boldface strings such as **if**, **id** etc...
2. These Symbols are nonterminals
 1. Uppercase letters early in the alphabet (like A, B, C)
 2. The letter S when it appears, is usually the *Start Symbol*
3. Lower case letters late in the alphabet chiefly *u*, *v*, *z* represent string of terminals

Notational Conventions

4. Uppercase letters late in the alphabet such as X, Y, Z represent grammar symbols that is either terminal or nonterminal
5. Lower case Greek letters α, β, γ represent strings of grammar symbols
6. If $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_n$ are all Productions with A on the LHS (known as *A-productions*), then, $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ ($\alpha_1, \alpha_2, \dots, \alpha_n$ are *alternatives* of A)
7. Unless otherwise stated , the LHS of the first production is the *start* nonterminal s

Use of Conventional Notation –

Example

Remember the previous Example:

$$\text{expr} \rightarrow \text{expr op expr}$$
$$\text{expr} \rightarrow (\text{expr})$$
$$\text{expr} \rightarrow - \text{expr}$$
$$\text{expr} \rightarrow \textit{id} \mid \textit{num}$$
$$\text{op} \rightarrow [+, -, *, /, \uparrow]$$

Using the notations described so far, this grammar can be written as:

$$E \rightarrow E A E \mid (E) \mid \textit{id} \mid \textit{num}$$
$$A \rightarrow + \mid - \mid * \mid \uparrow$$

Grammar for a Tiny Language

program ::= statement ; Statement

statement ::= assignStmt | ifStmt

assignStmt ::= id = expr ;

ifStmt ::= if (expr) stmt

expr ::= id | int | expr + expr

id ::= a | b | c | i | j | k | n | x | y | z

int ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Construct the CFG for the language having any number of a's over the set $\Sigma = \{a\}$.

As we know the regular expression for the above language is

r.e. = a^*

Production rule for the Regular expression is as follows:

$S \rightarrow aS$ rule 1

$S \rightarrow \epsilon$ rule 2

Now if we want to derive a string "aaaaaa", we can start with start symbols.

1. S
2. aS
3. aaS rule 1
4. aaaS rule 1
5. aaaaS rule 1
6. aaaaaS rule 1
7. aaaaaaS rule 1
8. aaaaaaε rule 2
9. aaaaaa

Example

Construct a CFG for a language $L = \{wcwR \mid w \in (a, b)^*\}$.

The string that can be generated for a given language is {aaciaa, bcb, abcba, bacab, abbcbbba,}

The grammar could be:

$S \rightarrow aSa$ rule 1

$S \rightarrow bSb$ rule 2

$S \rightarrow c$ rule 3

Now if we want to derive a string "abbcbbba", we can start with start symbols.

$S \rightarrow aSa$

$S \rightarrow abSba$ from rule 2

$S \rightarrow abbSbba$ from rule 2

$S \rightarrow abbcbbba$ from rule 3

Example :

Construct a CFG for the language $L = a^n b^{2n}$ where $n \geq 1$.

The string that can be generated for a given language is {abb, aabbbb, aaabbbbbbb....}.

The grammar could be:

$S \rightarrow aSbb \mid abb$

Now if we want to derive a string "aabbabb", we can start with start symbols.

$$S \rightarrow aSbb$$
$$S \rightarrow aabbabb$$

Regular Expressions and Grammars

EVERY construct that can be described by an RE can also be described by a grammar

Example : RE $(a|b)^*abb$ can be described as:

$$A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1 \quad A_1 \rightarrow bA_2 \quad A_2 \rightarrow bA_3 \quad A_3 \rightarrow \epsilon$$

Then why use RE for Lex & CFG for syntax analysis ?

- Lexical rules are frequently quite simple & we do not need powerful grammars to describe them
- RE's are more concise and easy to understand for tokens
- RE's provide for automatic construction of efficient Lexical Analyzers

Derivation



A way of showing how an input sentence is recognized with a grammar

Can also be thought of as a type of algorithm for recognizing the grammar

Remember the grammar:

1. $expr \rightarrow expr\ op\ expr$
2. $expr \rightarrow (expr)$
3. $expr \rightarrow -\ expr$
4. $expr \rightarrow id\ |\ num$
5. $op \rightarrow [+, -, *, /, \uparrow]$

Now let us derive a structure

$ctr + (a - 5) / 2$

$expr$
 $expr\ op\ expr$
 $expr + expr$
 $Expr + expr\ op\ expr$
 $expr + expr / expr$
 $expr + expr / num$
 $expr + (expr) / num$
 $expr + (expr - expr) / num$
 $id = id + (id - num) / num$

Derivations – Some definitions

Uses productions to derive more complex Expressions

The symbol:

- $\Rightarrow$ denotes “ derives in one step”.
- $\xRightarrow{*}$ denotes “ derives in **zero or more** steps” $(\alpha \xRightarrow{*} \beta)$
- $\xRightarrow{+}$ denotes “ derives in **one or more** steps” $(\alpha \xRightarrow{+} \beta)$

Given the grammar G with start symbol S , a string ω is part of the language $L(G)$, If and only if $S \xRightarrow{*} \omega$

Further, if $S \xRightarrow{*} \alpha$ and α contains

- Only terminals, then it is a –

Sentence of G

Sentential form of G

- nonterminals too, then it is a –

Derivation – Another Example

Consider the following Grammar a simple ; terminated expression consisting of interspersed numbers and +

1: $stmt \rightarrow expr ;$

2: $expr \rightarrow factor \text{ ' + ' } factor$

3: $\quad \quad | factor$

4: $factor \rightarrow \text{number}$

- Let us try and come up with a derivation to prove that an input sentence (like 1 + 4 ;) can be recognized from this grammar

$Stmt \Rightarrow expr ;$

$\Rightarrow factor + factor ;$

$\Rightarrow \text{Number} + factor ;$

$\Rightarrow \text{Number} + \text{Number} ;$

- The process is called a **Leftmost Derivation** because the **left most** nonterminal is **always** replaced

Leftmost Derivation



Start with the topmost symbol in the grammar

Replace the leftmost nonterminal in the partially parsed sentence with the equivalent production's RHS in each step

The operator



signifies left derivation

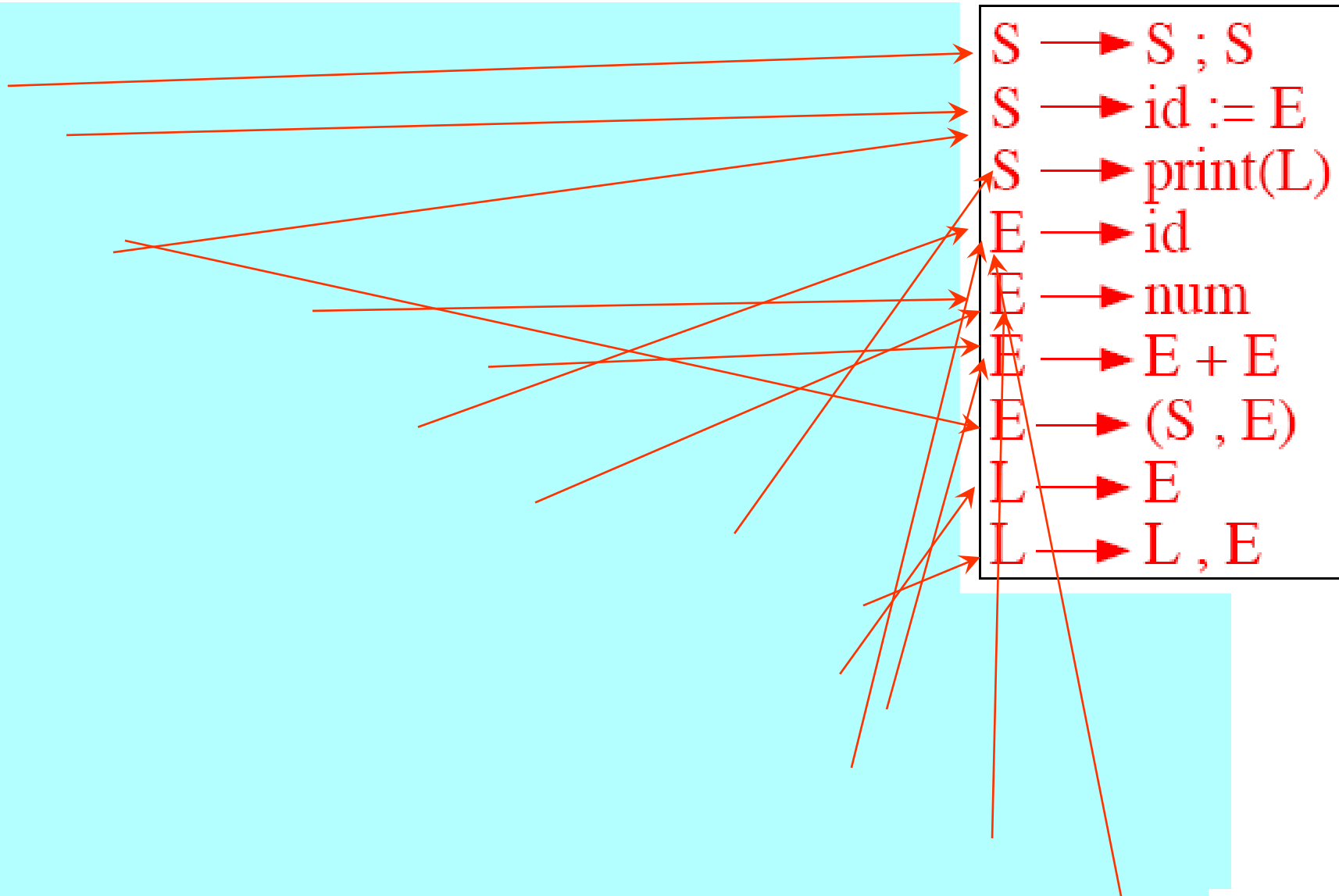
$\xRightarrow{L} Y$ means X derives Y by leftmost Derivation

X

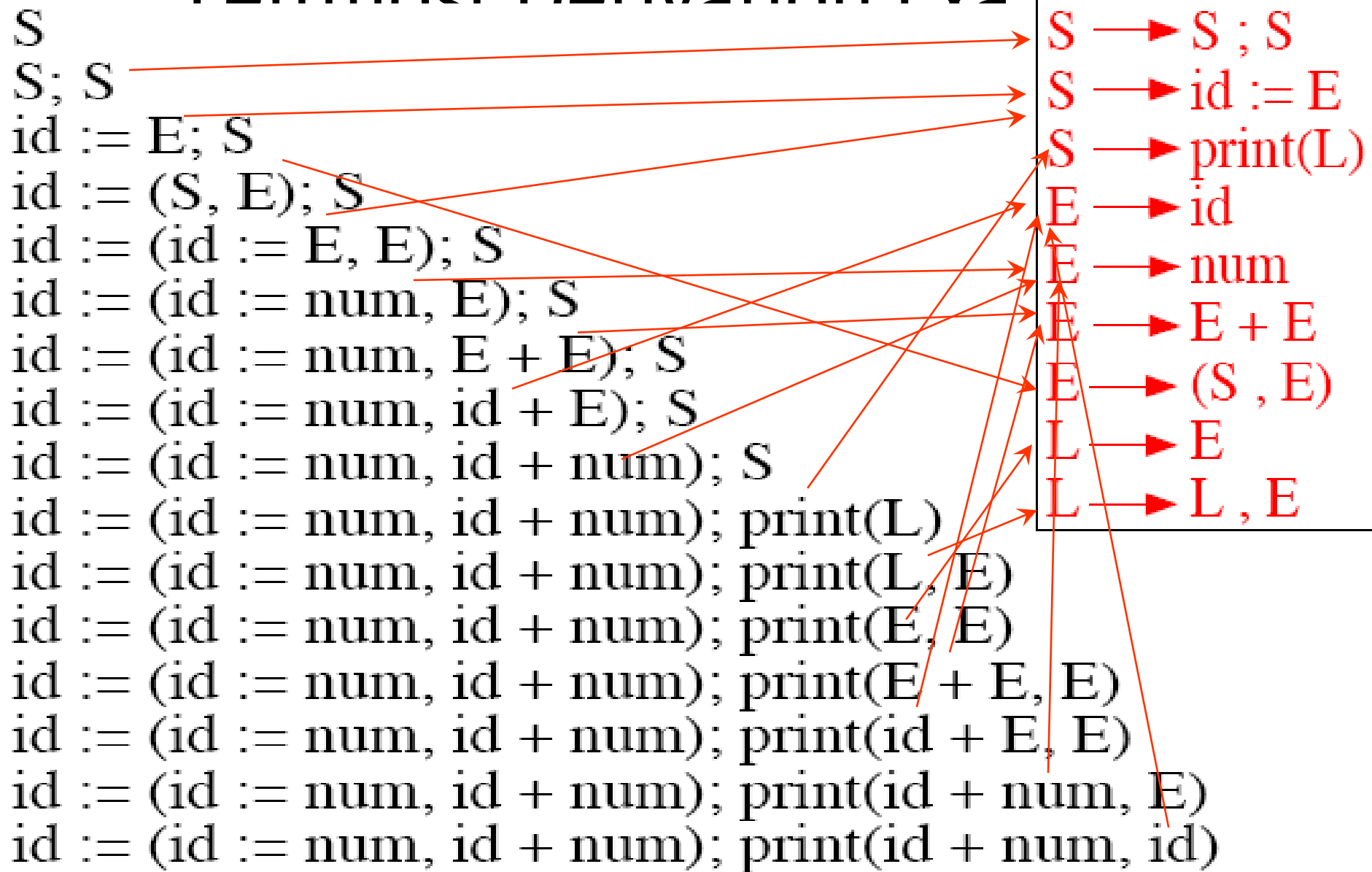
Each of the step (Partial Derivation) is called *Viable Prefix*

Portion of the viable prefix that is replaced at each step is called the *Handle*

Leftmost Derivation Example



Leftmost Derivation Example



Rightmost Derivation

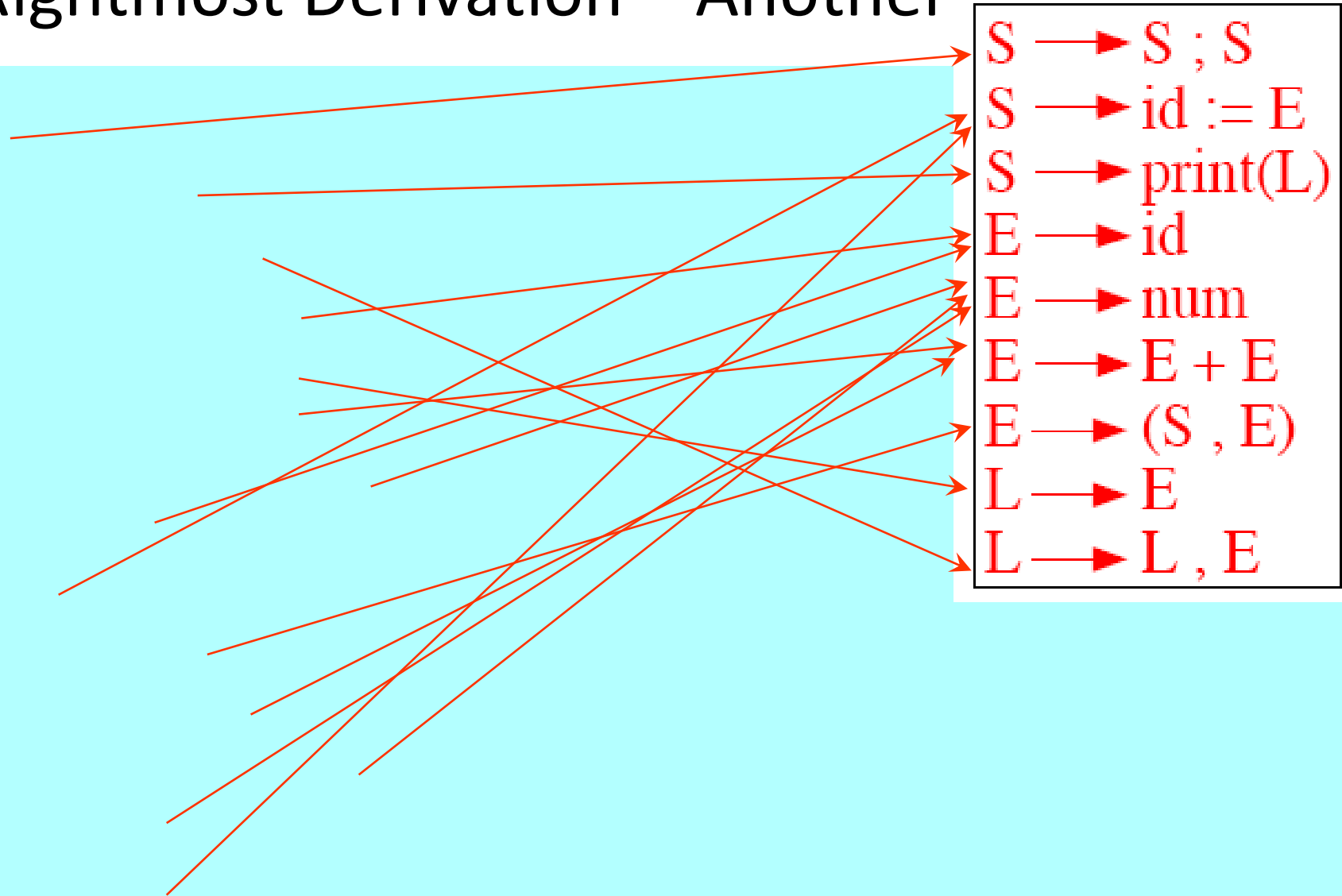


Replace the **rightmost** nonterminal in the partially parsed sentence $\xRightarrow{R}$ may signify Right Derivation

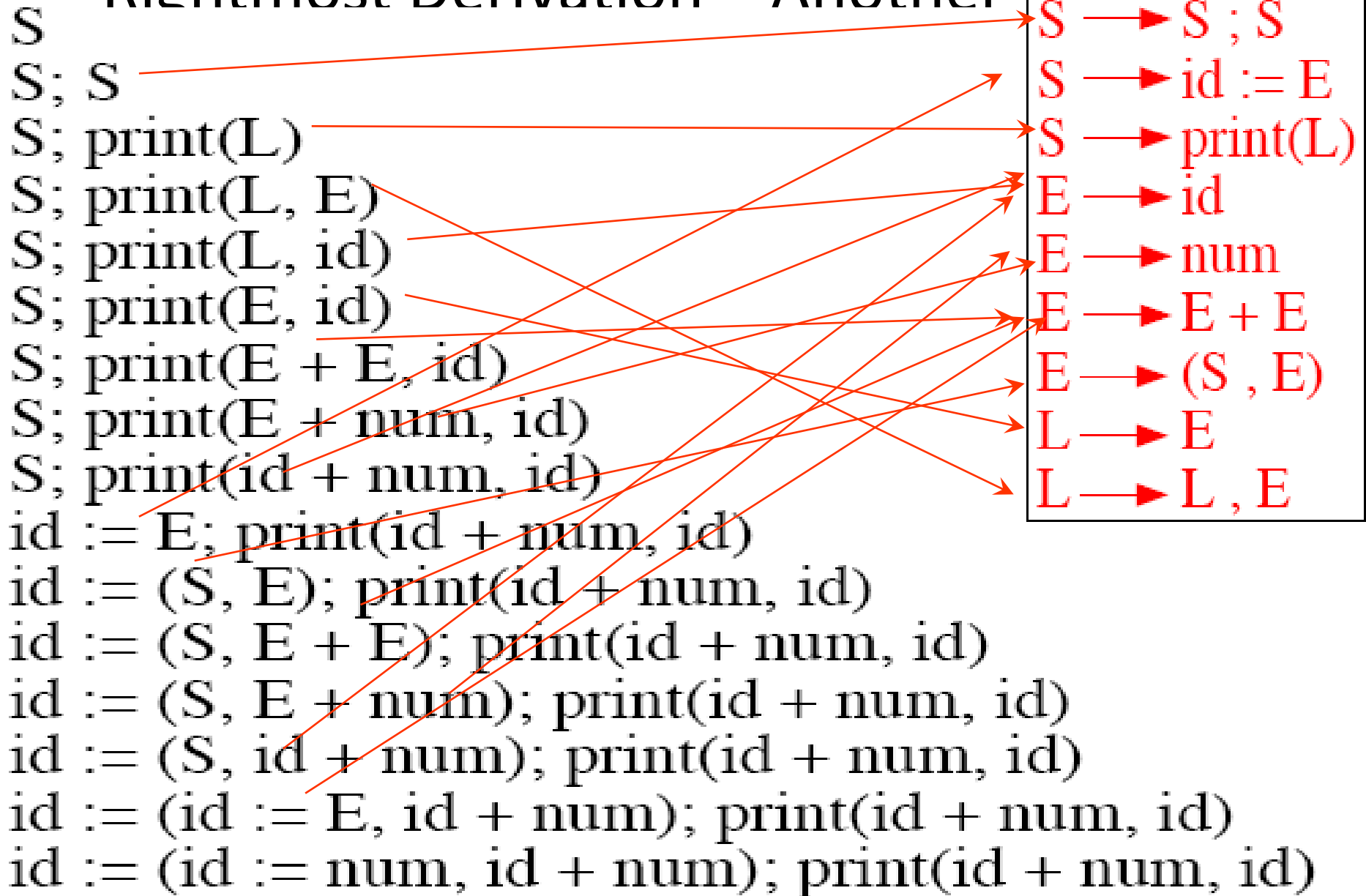
The operator
 $X \xRightarrow{R} Y$ means X derives Y by rightmost Derivation

It is possible to start with the start symbol and do rightmost derivation

Rightmost Derivation – Another Example



Rightmost Derivation – Another



Parse Tree

WHAT ?

A graphical representation for a derivation sequence showing how to derive the string of a language from grammar starting from Start symbol

WHY ?

Each stage of the compiler has two purposes:

- Detect and filter out some class of errors
- Compute some new information or translate the representation of the program to make things easier for later stages

Recursive structure of tree suits recursive structure of language definition

Parse Tree (Contd.)

HOW ?

Tree nodes represent symbols of the grammar (nonterminals or terminals) and tree edges represent derivation steps

Example:

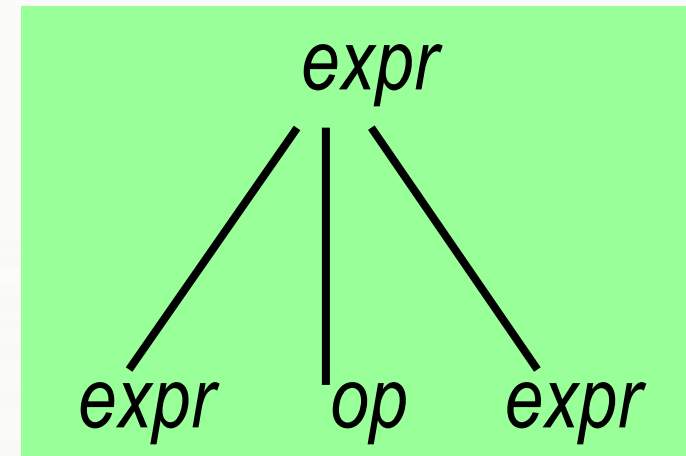
Given the production:

$expr \rightarrow expr\ op\ expr$

The corresponding Parse tree is :

It's Properties are:

1. The root is labeled by a start symbol
2. Each leaf is labeled by a token or by ϵ
3. Each interior node is labeled by a nonterminal



Parse Tree Example



Remember the grammar:

$\text{expr} \rightarrow \text{expr} + \text{expr}$

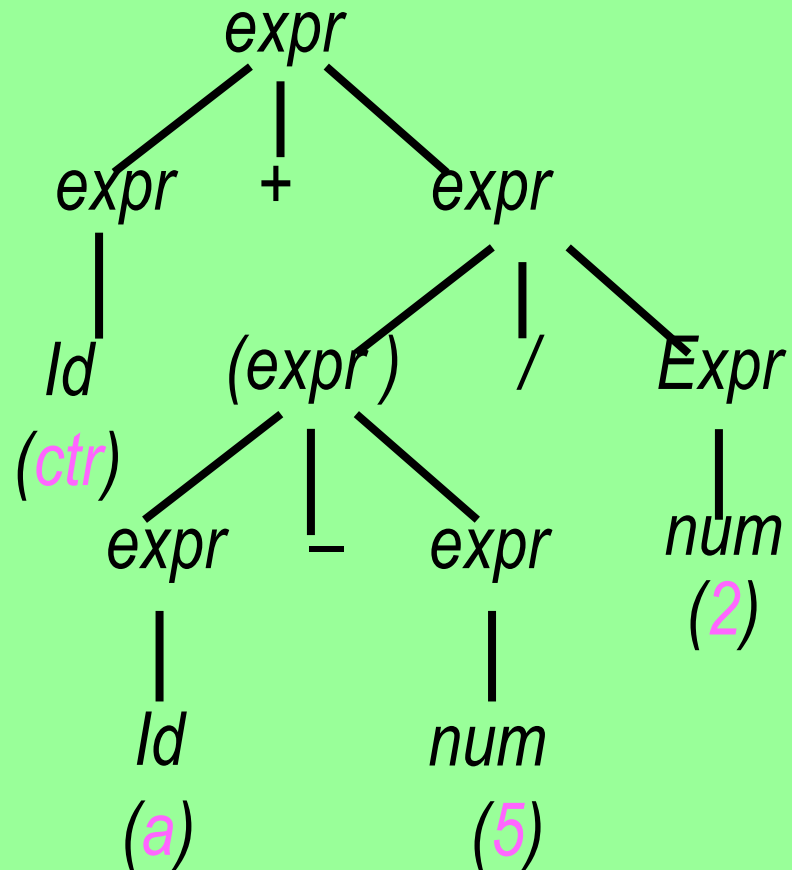
$\text{expr} \rightarrow \text{expr} + \text{expr} / \text{expr}$

$\text{expr} \rightarrow \text{expr} + (\text{expr}) / \text{expr}$

$\text{expr} \rightarrow \text{expr} + (\text{expr} - \text{expr}) / \text{expr}$

$\text{id} \rightarrow \text{id} + (\text{id} - \text{num}) / \text{num}$

The corresponding
Parse tree is



And the String derived from it

ctr + (a - 5) / 2

Using the derivation

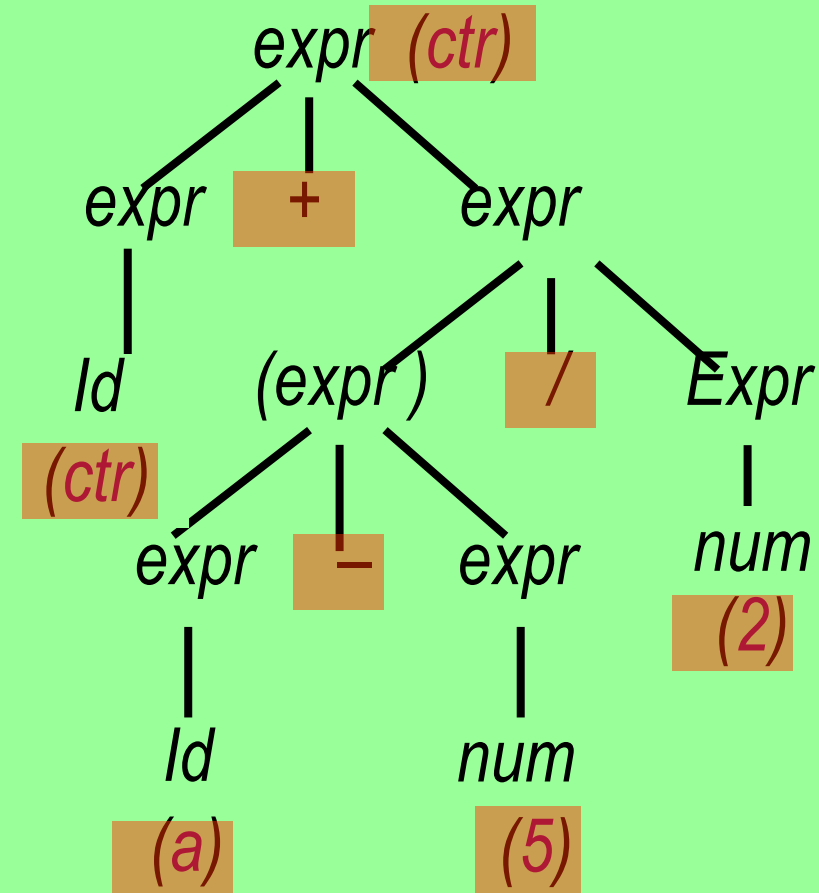
Parse Tree Example (Continued)

the *Yield* of the tree

- The string derived or generated from the nonterminal at the root of the tree
- Obtained by reading the leaves of the parse tree read from left to right

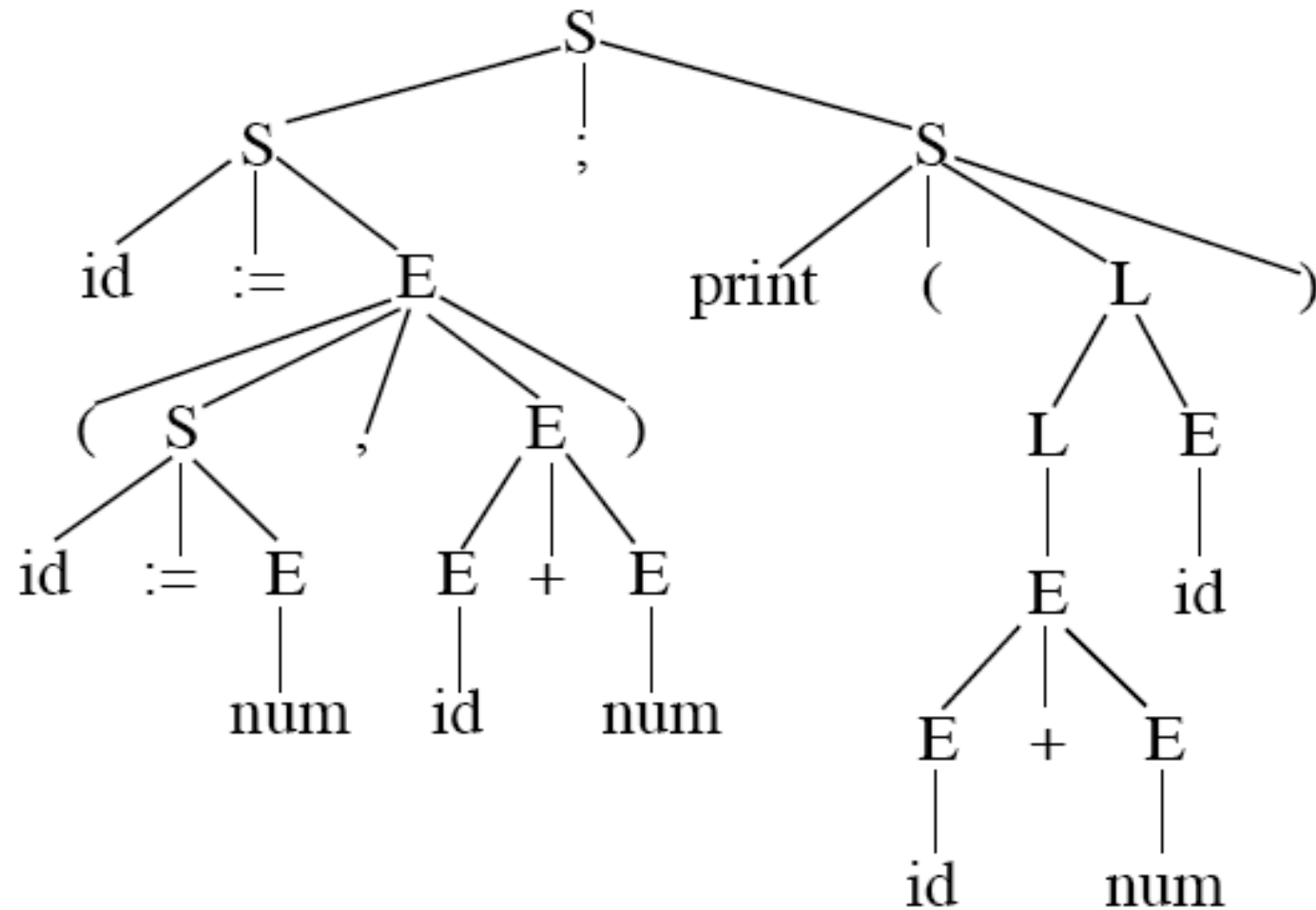
In this Example

$$\text{ctr} = \text{ctr} + (a - 5) / 2$$



Parse Tree – Another Example

$S \rightarrow S ; S$
 $S \rightarrow id := E$
 $S \rightarrow print(L)$
 $E \rightarrow id$
 $E \rightarrow num$
 $E \rightarrow E + E$
 $E \rightarrow (S, E)$
 $L \rightarrow E$
 $L \rightarrow L, E$



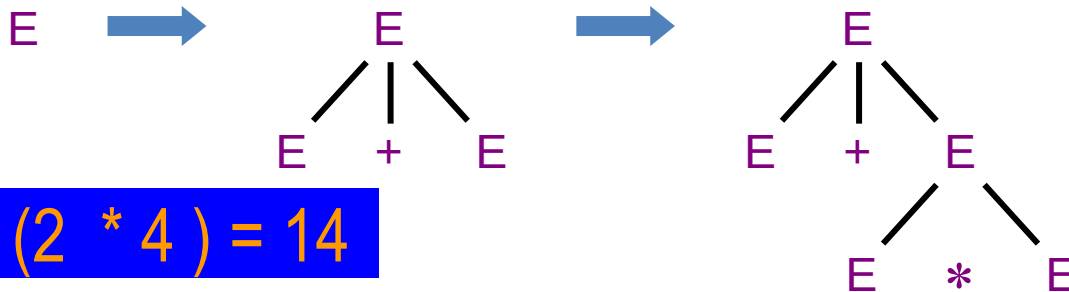
GRAMMAR
SENTENCE

$id := (id := num, id + num); print(id + num, id)$

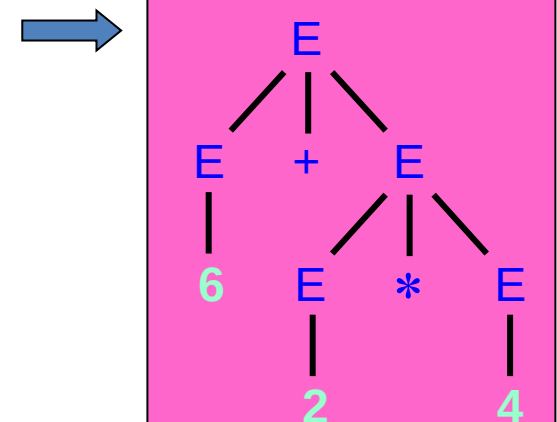
Another Derivation Example

Given the grammar : $E \rightarrow E + E \mid E * E \mid (E) \mid - E \mid \text{num}$

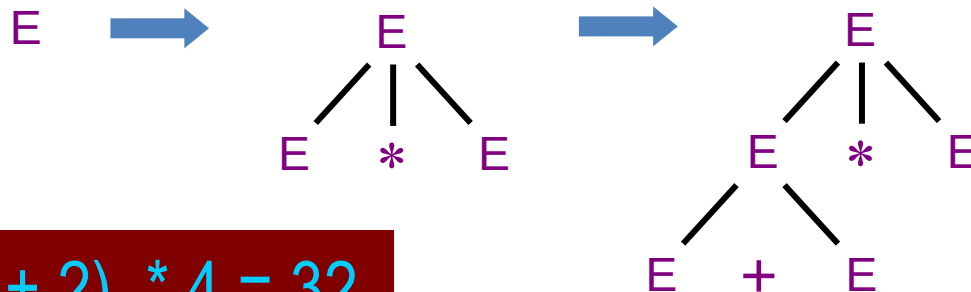
Find a derivation for the expression: $6 + 2 * 4$



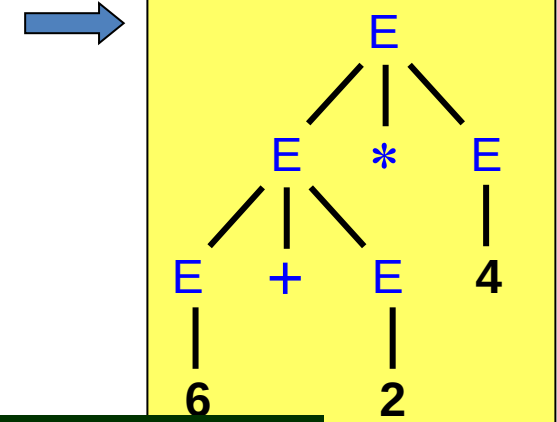
$$6 + (2 * 4) = 14$$



Which derivation tree is correct?



$$(6 + 2) * 4 = 32$$

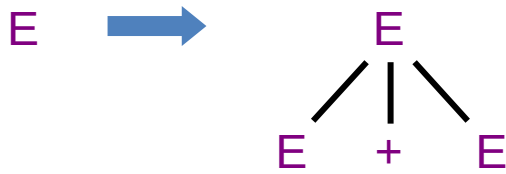


According to above grammar BOTH are correct!!!

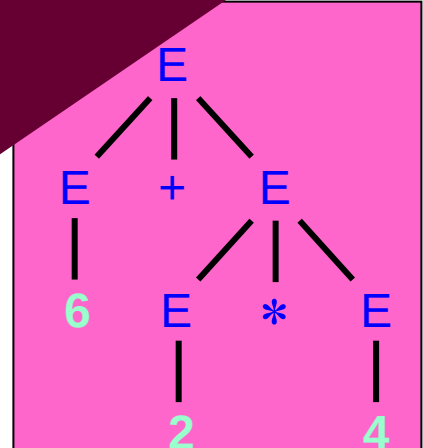
Another Derivation Example

Given the grammar : $E \rightarrow E + E \mid E * E \mid (E)$

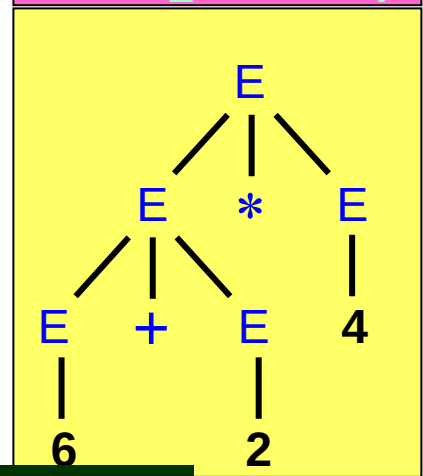
Find a derivation for the expression $6 + (2 * 4) = 14$



$6 + (2 * 4) = 14$



Which derivation tree is correct?



According to above grammar BOTH are correct!!!

A grammar that produces more than one parse tree for any input sentence is said to be an **ambiguous** grammar.

Ambiguity in Grammar



A grammar that produces more than one parse tree for any input sentence

- Happens because the derivation can be either
Leftmost or Rightmost

Can also happen because of the way the grammar is defined

Classic example — the if-then-else problem

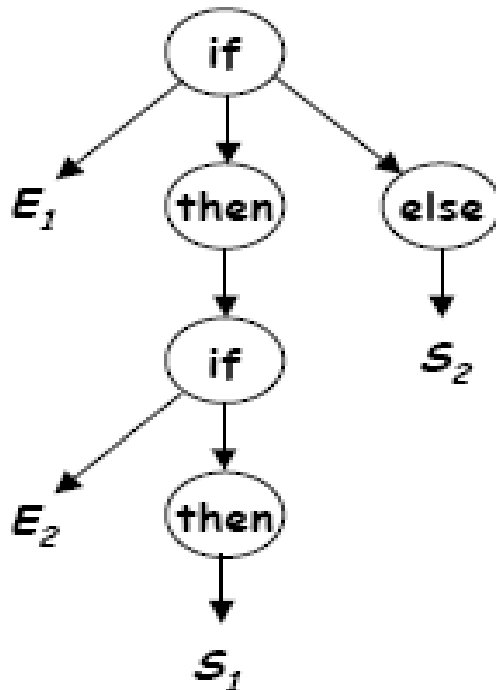
- $\text{Stmt} \rightarrow \text{if Expr then Stmt}$
- $| \text{if Expr then Stmt else Stmt}$
- $| \dots \text{other stmts} \dots$

Ambigu

Stmt $\rightarrow$ If Expr then Stmt
| if Expr then Stmt else Stmt
| ... other stmts

This sentential form has two derivations

if *Expr1* then if *Expr2* then *Stmt1* else *Stmt2*



production 2,
then production 1

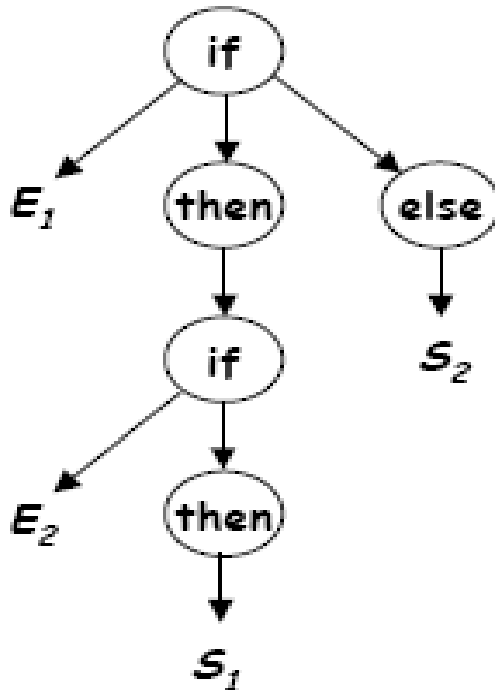
...

Stmt $\rightarrow$ If Expr then Stmt
 | if Expr then Stmt else Stmt
 | ... other stmts

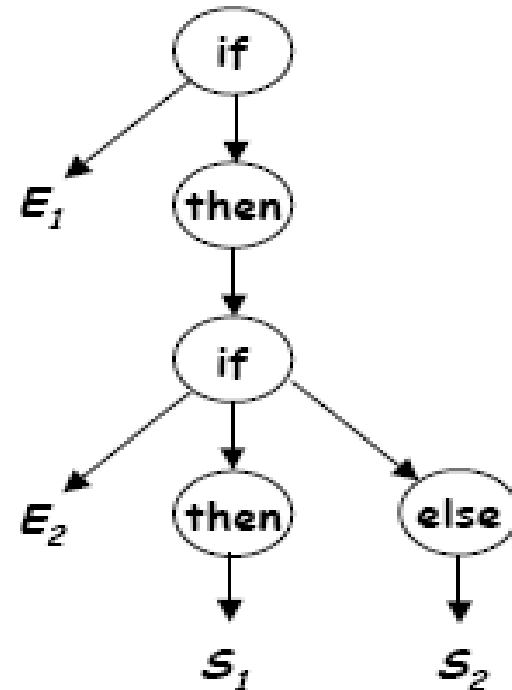
Ambiguous IF

This sentential form has two derivations

if *Expr1* then if *Expr2* then *Stmt1* else *Stmt2*



production 2,
then production 1



production 1,
then production 2

Check whether the given grammar G is ambiguous or not.

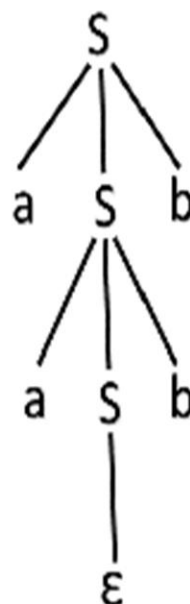
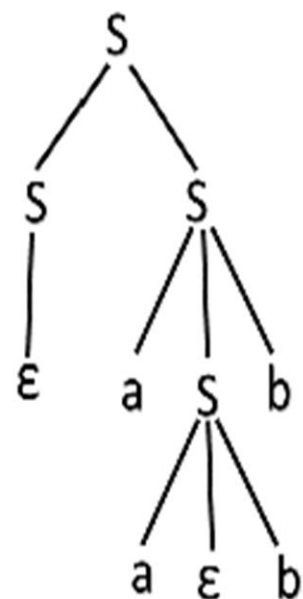
1. $S \rightarrow aSb \mid SS$
2. $S \rightarrow \epsilon$

Check whether the given grammar G is ambiguous or not.

1. $A \rightarrow AA$
2. $A \rightarrow (A)$
3. $A \rightarrow a$

Solution:

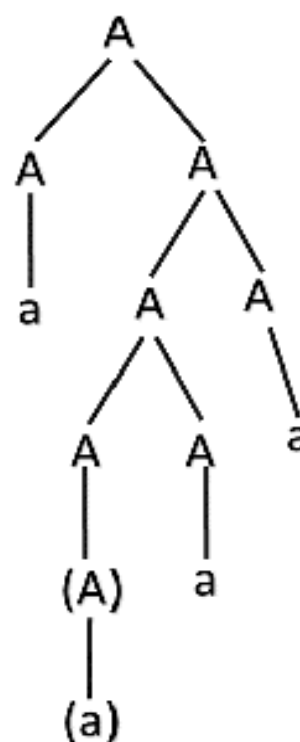
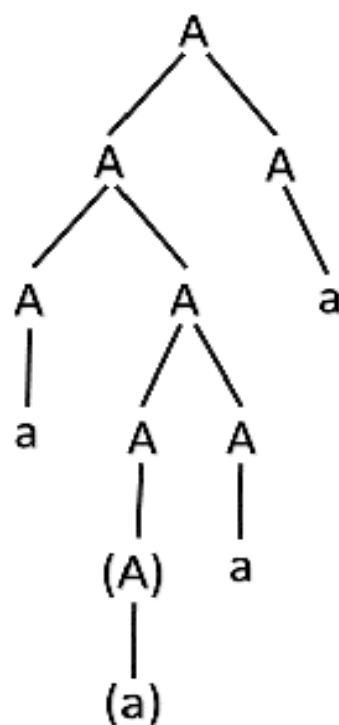
For the string "aabb" the above grammar can generate two parse trees



Since there are two parse trees for a single string "aabb", the grammar G is ambiguous.

Solution:

For the string "a(a)aa" the above grammar can generate two parse trees:



Since there are two parse trees for a single string "a(a)aa", the grammar G is ambiguous.

Recursion In Grammars

Very Common in grammars

Generally 3 types of recursions are Possible:

1. $A ::= Ab$ (Left Recursion)
2. $B ::= cB$ (Right Recursion)
3. $C ::= dCf$ (Middle Recursion or Self-embedding)

Facts:

*If a grammar contains **no middle recursion** then the language it generates is regular.*

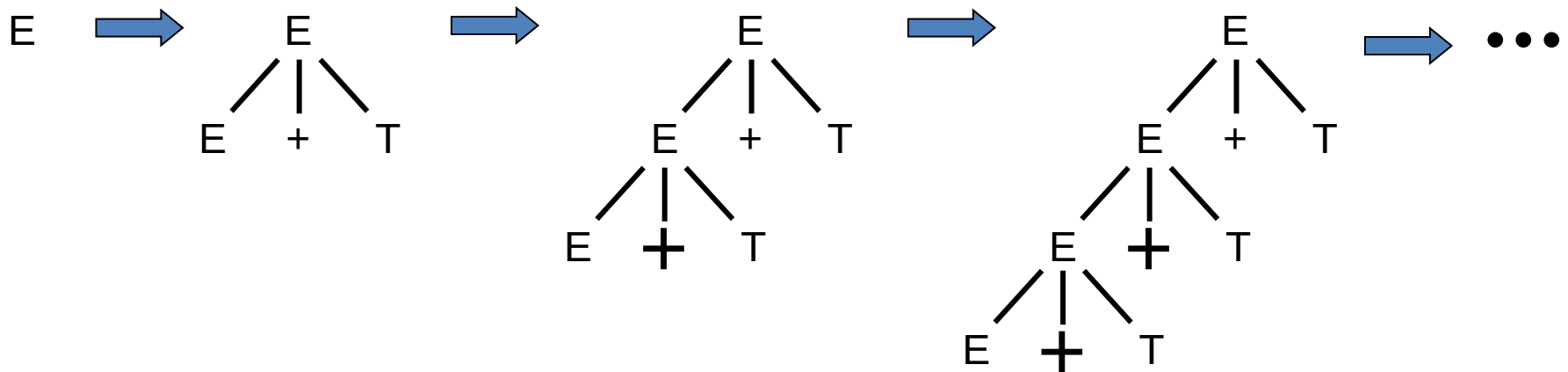
If there is no recursion at all in a grammar, then it is finite and therefore regular.

Left Recursion

Consider the grammar:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

A top-down parser might loop forever when parsing an expression using this grammar



- A grammar that has at least one production of the form $A \Rightarrow A\alpha$ is a **left recursive** grammar

Top Down parsing cannot handle left-recursive grammars

Eliminating left Recursion

First group the A productions as

$$A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \mid$$

$$A \rightarrow A\alpha_1 \mid$$

Where no β_i begins with an A.

Then replace the A-productions by $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \varepsilon$$

This eliminates all immediate left recursions from A and A' productions

BUT

It does not eliminate left recursions involving derivations of two or more steps

Left Recursion (Contd.)

Left-recursion can often be eliminated by rewriting the grammar

This left-recursive grammar:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Can be re-written to eliminate the immediate left recursion:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Properties of Grammars – A Summary

Epsilon Productions

- Has at least one production of the form “ $E \rightarrow \varepsilon$ ”,

Right(/ Left) Linear Grammar:

- Grammars where all the RHS of all productions has at most one non-terminal and that nonterminal appears rightmost (/ Leftmost.)
- $E \rightarrow x E$ or “ $E \rightarrow x$ ”.

Left-Recursive Grammar:

- A grammar that can generate “ $E \rightarrow E + X$ ” (for example).

Similarly, “Right-recursive grammars.”

Ambiguous Grammar

- More than one parse tree is possible for a specific sentence.

Limitations of CFG

CFGs cannot express context conditions

For Example:

- 1. Every name must be declared before it is used*
- 2. The operands of an expression must have compatible types*

Possible solutions

Use context-sensitive grammars

- Too complicated

Check context conditions during semantic analysis

Syntax Analysis

Problem Statement:

To find a derivation sequence in a grammar **G** for the input token stream

or

To conclude that none exists

Solution

- Build parse tree for the string (of tokens) based on grammar rules
- The process is known as **P A R S I N G**

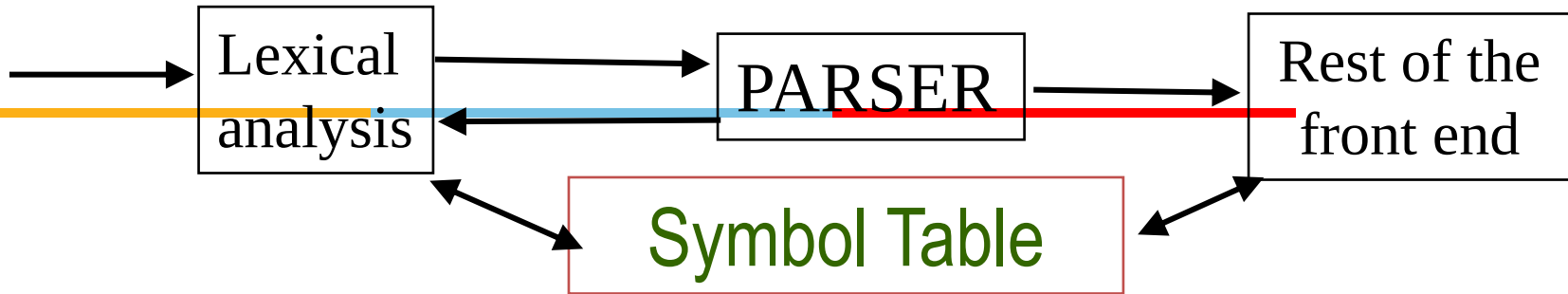
PARSING



Constructs the syntax tree for a given sequence of tokens using grammar rules and productions such that:

1. The top node is labeled with the start symbol of the grammar
2. Leaf nodes are labeled with terminals and inner nodes with nonterminals
3. the children of the inner node labeled N correspond to the members of an alternative of N, in the same order as they occur in that alternative
4. The terminals labeling the leaf nodes correspond to the sequence of tokens, in the same order they occur in the input

The Role of a Parser



The *parser continues the process of translation and validation* started by the Lexical Analyzer:

1. Analyzing the phrase structure of the program,
2. Adding hierarchical (tree) structure to the flat stream of tokens produced by the lexical Analyzer,
3. Outputting a data structure conveying the program's syntax to subsequent compile phases,
4. Reporting errors when the program does not match the syntactic structure of the source language

FIRST & FOLLOW

innovate

achieve

lead

The two functions represent a set of terminal symbols, that aid us in constructing Predictive parser by helping us fill the Parsing Table with valid entries

Set of terminal symbols yielded by *FOLLOW* function can also be used in error recovery

FIRST – if α is the string of grammar symbols then *FIRST* (α) is the set of terminals that begin the strings derived from α

(If $\alpha \xRightarrow{*} \varepsilon$, then ε is also in *FIRST* (α))

FOLLOW (*A*) – the set of terminals *a* that can appear immediately to the right of *A* such that there exists a derivation of the form

$S \xRightarrow{*} \alpha A a \beta$ for some α and β

FIRST & FOLLOW (Contd.)

- The set of terminal symbols (including ϵ) that can appear at the far left of any parse tree derived from a particular nonterminal is the FIRST set of that nonterminal
- The set of terminal symbols (including ϵ) That can follow a nonterminal in some derivation or the other, is called the FOLLOW set of that nonterminal
- A set of rules are followed to compute the FIRST and FOLLOW set
- These sets will be used in creating Parsing table

Rules to Create FIRST

FIRST rules:

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$
2. If $X \rightarrow \varepsilon$, then $\varepsilon \in \text{FIRST}(X)$
3. If $X \rightarrow aABC$, then $a \in \text{FIRST}(X)$
4. If $X \rightarrow ABCD$, then $\text{FIRST}(A) \in \text{FIRST}(X)$.
- 4a. Further, If $A \rightarrow \varepsilon$, in the above production, then $\text{FIRST}(B) \in \text{FIRST}(X)$ [& so on recursively]

GRAMMAR:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

SETS:

$$\text{FIRST}(\text{id}) = \{\text{id}\}$$
$$\text{FIRST}(*) = \{*\}$$
$$\text{FIRST}(+) = \{+\}$$
$$\text{FIRST}(()) = \{($$
$$\text{FIRST}(()) = \{)\}$$
$$\text{FIRST}(E') = \{\cancel{\varepsilon}\{+, \varepsilon\}$$
$$\text{FIRST}(T') = \{\cancel{\varepsilon}\{*, \varepsilon\}$$
$$\text{FIRST}(F) = \{(, \text{id}\}$$
$$\text{FIRST}(T) = \text{FIRST}(F) = \{(, \text{id}\}$$
$$\text{FIRST}(E) = \text{FIRST}(T) = \{(, \text{id}\}$$

$\text{FIRST}(E') = \{+, \epsilon\}$

$\text{FIRST}(T') = \{*, \epsilon\}$

$\text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(T) = \{ (, \text{id} \}$

$\text{FIRST}(E) = \{ (, \text{id} \}$

Rules to Create FOLLOW

FOLLOW rules:

1. If **S** is the start symbol, then $\$ \in \text{FOLLOW}(S)$

2. If there is a production $A \rightarrow \alpha B \beta$, then,
EVERYTHING in $\text{FIRST}(\beta)$ except ϵ is placed
in $\text{FOLLOW}(B)$

3. If there is a production $A \rightarrow \alpha B$ then,
EVERYTHING in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$

3a. If there is a production $A \rightarrow \alpha B \beta$ where
 $\text{FIRST}(\beta)$ contains ϵ (i.e. $\beta \xRightarrow{*} \epsilon$) then,
EVERYTHING in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$

A and B are non-terminals,
 α and β are strings of grammar symbols

GRAMMAR:

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid \text{id}$

SETS:

$\text{FOLLOW}(E) = \{\cancel{\$}\{), \$\}$

$\text{FOLLOW}(E') = \{), \$\}$

$\text{FOLLOW}(T) = \{), \cancel{\$}\{+,), \$\}$

$\text{FOLLOW}(T') = \{ +,), \$\}$

$\text{FOLLOW}(F) = \{\cancel{+}, \cancel{), \$\}$
 $\{+, *,), \$\}$

Find First and Follow

- **CFG**

$S \rightarrow EF$

$E \rightarrow g \mid \epsilon$

$F \rightarrow f \mid \epsilon$

- **CFG**

$S \rightarrow EFG$

$E \rightarrow g \mid o$

$F \rightarrow f \mid \epsilon$

$G \rightarrow k \mid l$

Symbol	First	Follow
E	g €	f €
F	f €	\$
S	g €	\$

Symbol	First	Follow
E	g o	f €
F	f €	k l
G	k l	\$
S	g o	\$

Example of first and follow function

How to find the first and follow functions for the given CFG?

$S \rightarrow aXZh$

$X \rightarrow cY$

$Y \rightarrow bY / \epsilon$

$Z \rightarrow EF$

$E \rightarrow g / \epsilon$

$F \rightarrow f / \epsilon$

First Functions

- $\text{First}(S) = \{ a \}$
- $\text{First}(X) = \{ c \}$
- $\text{First}(Y) = \{ b, \epsilon \}$
- $\text{First}(Z) = \{ \text{First}(E) - \epsilon \} \cup \text{First}(F) = \{ g, f, \epsilon \}$
- $\text{First}(E) = \{ g, \epsilon \}$
- $\text{First}(F) = \{ f, \epsilon \}$

Follow Functions

- $\text{Follow}(S) = \{ \$ \}$
- $\text{Follow}(X) = \{ \text{First}(Z) - \epsilon \} \cup \text{First}(h) = \{ g, f, h \}$
- $\text{Follow}(Y) = \text{Follow}(X) = \{ g, f, h \}$
- $\text{Follow}(Z) = \text{First}(h) = \{ h \}$
- $\text{Follow}(E) = \{ \text{First}(F) - \epsilon \} \cup \text{Follow}(Z) = \{ f, h \}$
- $\text{Follow}(F) = \text{Follow}(Z) = \{ h \}$

Example of first and follow function with Left Recursion

How to find the first and follow functions for the given CFG with Left Recursive production rules.?

$S \rightarrow X$

$X \rightarrow aY / Xd$

$Y \rightarrow b$

$C \rightarrow t$

The given grammar is left recursive. So, we first remove the left recursion from the given grammar.

After the elimination of left recursion, we get the following grammar.

$$S \rightarrow X$$
$$X \rightarrow aYX'$$
$$X' \rightarrow dX' / \epsilon$$
$$Y \rightarrow b$$
$$C \rightarrow t$$

The first and follow functions are described below;

First Functions

- $\text{First}(S) = \text{First}(X) = \{ a \}$
- $\text{First}(X) = \{ a \}$
- $\text{First}(X') = \{ d, \epsilon \}$
- $\text{First}(Y) = \{ b \}$
- $\text{First}(C) = \{ t \}$

Follow Functions-

- $\text{Follow}(S) = \{ \$ \}$
- $\text{Follow}(X) = \text{Follow}(S) = \{ \$ \}$
- $\text{Follow}(X') = \text{Follow}(X) = \{ \$ \}$
- $\text{Follow}(Y) = \{ \text{First}(A') - \epsilon \} \cup \text{Follow}(A) = \{ d, \$ \}$
- $\text{Follow}(C) = \text{NA}$

Example of first and follow function with Left Recursion

How to find the first and follow functions for the given CFG with Left Recursive production rules.?

$$X \rightarrow X + Y / Y$$
$$Y \rightarrow Y d F / F$$
$$F \rightarrow (X) / id$$

Solution-

The given grammar is left recursive. So, we first remove the left recursion from the given grammar.

After the elimination of left recursion, we get the following grammar.

$$X \rightarrow YX'$$

$$X' \rightarrow + YX' / \epsilon$$

$$Y \rightarrow FY'$$

$$Y' \rightarrow d FY' / \epsilon$$

$$F \rightarrow (X) / \text{id}$$

The first and follow functions are described below;

First Functions

- $\text{First}(X) = \text{First}(Y) = \text{First}(F) = \{ (, \text{id} \}$
- $\text{First}(X') = \{ + , \epsilon \}$
- $\text{First}(Y) = \text{First}(F) = \{ (, \text{id} \}$
- $\text{First}(Y') = \{ d , \epsilon \}$
- $\text{First}(F) = \{ (, \text{id} \}$

Follow Functions

- $\text{Follow}(X) = \{ \$,) \}$
- $\text{Follow}(X') = \text{Follow}(E) = \{ \$,) \}$
- $\text{Follow}(Y) = \{ \text{First}(X') - \epsilon \} \cup \text{Follow}(X) \cup \text{Follow}(X')$
 $= \{ + , \$,) \}$
- $\text{Follow}(Y') = \text{Follow}(Y) = \{ + , \$,) \}$
- $\text{Follow}(F) = \{ \text{First}(Y') - \epsilon \} \cup \text{Follow}(Y) \cup \text{Follow}(Y')$
 $= \{ d , + , \$,) \}$

References

<https://www.javatpoint.com/context-free-grammar>

<https://t4tutorials.com/rules-of-first-and-follow-in-predictive-parsing/>

IMP Note to Self



Thank you